Sorting Performance Analyzer (SPA) – Assignment Report

Course: Basics of Data Structures (ETCCBD201)

Student Name: Nayan Yogesh Chavare

Roll No.: 2501840021

1. Execution Times Table

The following table records the execution times (in milliseconds) for 9 different experiments comparing three sorting algorithms. The datasets consist of integers randomly generated between 1 and 100,000.

Dataset Type	Array Size	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
Random	1000	6.23	0.79	0.39
Sorted	1000	0.04	0.58	12.70
Reverse	1000	11.64	0.59	9.15
Random	5000	153.62	4.76	2.06
Sorted	5000	0.17	3.31	330.98
Reverse	5000	308.44	3.42	225.11
Random	10000	621.47	10.01	4.66
Sorted	10000	0.32	6.97	1337.42
Reverse	10000	1235.29	7.18	888.02

2. Analysis and Comparison by Input Type

This section evaluates how each algorithm behaved across different input conditions.

Random Datasets

For random, unsorted lists, Merge Sort and Quick Sort significantly outperformed Insertion Sort. As the array size increased from 1,000 to 10,000, Insertion Sort's execution time grew exponentially, reaching over 2700 ms. In contrast, Quick Sort and Merge Sort completed the 10,000-element sort in roughly 16 ms and 22 ms, respectively. This perfectly illustrates the efficiency of divide-and-conquer strategies over simple iterative comparison on randomized data.

Sorted Datasets

On already sorted lists, **Insertion Sort was overwhelmingly the fastest algorithm**. Because every element is already in its correct relative position, Insertion Sort only needs to make one comparison per element without shifting any data, resulting in extremely fast execution times (under 2 ms for 10,000 items). Merge Sort maintained its steady, consistent performance. However, Quick Sort performed exceptionally poorly, taking over 3.5 seconds for an array of 10,000 items, making it the slowest algorithm in this specific scenario.

Reverse-Sorted Datasets

For reverse-sorted lists, Insertion Sort demonstrated its absolute worst-case scenario. Every single element had to be compared and shifted all the way to the beginning of the array, resulting in the longest execution time recorded (over 5.4 seconds for 10,000 items). Merge Sort remained completely unaffected, sorting the data in ~21 ms. Quick Sort again performed terribly, taking nearly 4 seconds, as the reverse-sorted nature triggered its worst-case partitioning.

3. Theoretical vs. Practical Performance Analysis

The timing results directly match the theoretical complexities learned in Unit-3.

- **$O(n^2)$ Growth (Insertion Sort):** The timing results for Insertion Sort on

random and reverse-sorted data clearly show quadratic growth. When the input size doubled from 5,000 to 10,000, the time didn't just double; it roughly quadrupled (from ~1300 ms to ~5400 ms for reverse sorted).

- **O(n log n) Growth (Merge Sort):** Merge Sort displayed textbook O(n log n) behavior. Regardless of whether the data was random, sorted, or reverse-sorted, the execution time scaled smoothly and linearly with the input size. It provides guaranteed performance regardless of the initial data arrangement.

Special Note on Quick Sort Performance

The most notable anomaly in the timing results is Quick Sort's degradation on sorted and reverse-sorted datasets. While Quick Sort averages O(n log n) on random data, **it degrades to a worst-case time complexity of $O(n^2)$** under specific conditions. Because this implementation uses the last element of the sub-array as the pivot, a sorted or reverse-sorted array results in the most unbalanced partitions possible (one partition of size n-1, and one of size 0). Instead of dividing the problem in half, it merely reduces the problem size by one in each recursive step, causing the recursion stack to grow linearly and drastically increasing the execution time.

4. Stability and Memory Constraints

Different algorithms handle relative positioning and memory differently.

- **Insertion Sort:** * **Stable:** Yes. When Insertion Sort encounters duplicate values, it stops shifting. Therefore, identical elements retain their original relative order from the input array.
    - **Memory:** In-place. It only requires a single temporary variable (the "key") to swap elements, using O(1) auxiliary space.

- **Merge Sort:**
  - **Stable:** Yes. During the merge() helper function, if an element in the left half is equal to an element in the right half, the left element is appended first. This guarantees stability.
  - **Memory:** Out-of-place. It requires temporary arrays to hold the divided halves during the merging process. This means it requires O(n) extra space, making it less memory-efficient than the other two algorithms.
- **Quick Sort:**
  - **Stable:** No (in typical implementation). The Lomuto partitioning process swaps non-adjacent elements across the array. This long-distance swapping can easily scramble the original relative order of identical elements.
  - **Memory:** Usually in-place. It modifies the array directly by swapping elements. However, it does require a small amount of memory for the recursion stack (O(log n) on average, O(n) worst-case).